

LightCCI 主方法与 Network 架构补全 Codex Plan

LightCCI 主方法与 Network 架构补全 Codex Plan

0. 当前代码状态与严重问题

当前 code.zip 已经包含 mignet_ce/pij/compare/ 下的一组 compare 方法雏形，也已经把旧 PIJ 方法移动到 mignet_ce/pij/legacy/。但是当前代码仍然存在两个致命缺口：

1. 没有真正实现 **light_cci network method**。
 - 当前 mignet_ce/networks/ 下没有 light_cci.py。
 - 当前 mignet_ce/networks/registry.py 的 NETWORK_BUILDERS 里没有 "light_cci"。
 - 当前 mignet_ce/config.py 的 NETWORK_METHODS 里没有 "light_cci"。
 - 因此如果用户运行 --network-method light_cci，配置校验会直接失败。
 - 如果用户改用旧的 clean_expression_cci_mix、clean_grn_cci_mix 或 legacy network 来跑 compare 方法，那么实验实际用的仍然不是 LightCCI network 架构。
2. 缺少原主方法入口。
 - 目前 compare 矩阵主要是 30 个 feature x PIJ method 消融格子。
 - 但用户原本的主方法是：LightCCI 图 -> Laplacian-HKS 特征 + SR + spatial 预成本 -> sparse semi-relaxed OT -> PIJ。
 - 这个方法不应该被塞进 30 格消融矩阵里，但必须在 mignet_ce/pij/compare/ 下作为一个独立代码文件实现和注册。

本次 Codex 的任务不是继续补小修小改，而是必须先把这两个主干缺口补上：

- 新增并注册 light_cci network method。
- 新增并注册 LightCCI 主方法 compare_main_lap_sr_spatial_sot。

1. LightCCI 的定义必须重新落到 network 层

LightCCI 不是旧网络的一个参数，也不是 PIJ 里面临时读取 CCI_total.npz。它必须是 mignet_ce/networks/ 下的一个新 network method。

LightCCI 的核心定义如下：

- **gene 层级**：节点是 gene，边来自 gene x gene 的 GRN。这里不是 GGI，也不是 CCI，也不是 LR pair 通讯。第一版把 GRN 权重取非负强度，例如使用 abs(weight) 或只保留正权重，供后续 Laplacian-HKS 和 NMF 使用。
- **spot 层级**：节点是 spot，边来自 spot 层级 COMMOT/CCI 输出，例如 *_CCI_total.npz，并用 *_index.tsv 对齐节点顺序。
- **domain 层级**：节点是 domain，边来自对应 domain 层级 COMMOT/CCI 输出，例如 seurat_k40、seurat_k150、louvain_k40、louvain_k150 或 spatial domain 层级下的 *_CCI_total.npz。
- **表达矩阵不是 LightCCI network 的核心边来源**。表达数据可以用于取节点、坐标、shared genes、以及 compare 矩阵中的 E 特征，但不能把 LightCCI 重新变成 expression/GRN/CCI mix 旧网络。

因此，LightCCI 的职责是：给每个 organ、time point、layer 输出一个轻量图对象，其中 cell/spot/domain 使用 CCI，gene 使用 GRN。

2. 必须先检查 sample data 中 COMMOT/CCI 输出长什么样

本机 sample data 已经在来源文件中提供，例如 E1S1_domain_factory_sample.zip。Codex 需要先在本地 sample data 中检查这些文件的结构，而不是凭空猜路径：

- cci/<layer>/<sample>_CCI_total.npz
- cci/<layer>/<sample>_index.tsv
- cci/<layer>/<sample>_COMMOT_lr_pairs.tsv
- cci/<layer>/<sample>_COMMOT_by_LR/*.npz
- grn/<layer>/<sample>/grn_edges.csv
- 如存在 gene 层或 group-level GRN，也需要检查对应 GRN 文件实际位置和字段。

建议先用下面的本机检查脚本确认 .npz 的 shape、nnz 和 index 对齐：

```
python - <<'PY'
from pathlib import Path
import zipfile

sample_zip = Path('/mnt/data/E1S1_domain_factory_sample.zip')
with zipfile.ZipFile(sample_zip) as zf:
    names = zf.namelist()
    for pat in ['CCI_total.npz', '_index.tsv', 'COMMOT_by_LR', 'grn_edges.csv']:
        hits = [n for n in names if pat in n]
        print('\n#', pat, 'hits=', len(hits))
        for h in hits[:10]:
            print(h)
```

PY

正式运行时不使用 /mnt/data，而是服务器数据根目录，例如：

/home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory

Codex 要写的是服务器可运行代码，但本机 sample data 可以用于理解文件格式和做轻量测试。

3. 新增 mignet_ce/networks/light_cci.py

新增文件：

mignet_ce/networks/light_cci.py

新增类：

```
class LightCCINetworkBuilder:
    network_method = "light_cci"
```

它需要实现和现有 network builder 一致的接口：

```
def build_pair_context(self, organ, pair, cfg, resolver) -> NetworkContext:
    ...
```

3.1 对 spot/domain 层读取 CCI 图

对于 spot、seurat_*、louvain_*、spatial_domain_* 等非 gene 层：

1. 优先读取：

```
paths.cci_total
paths.cci_index
```

2. 如果 paths.cci_total 不存在，再 fallback 到：

```
paths.cci_manifest
paths.cci_lr_dir/*.npz
```

把所有 LR pair 的 COMMOT 矩阵求和得到 total adjacency。

3. 使用 `paths.cci_index` 对齐节点顺序，不允许假设 `.npz` 行列顺序等于 `h5ad obs` 顺序。

4. 输出非负稀疏邻接矩阵：

$A_{\{layer,t\}} \in \mathbb{R}_{+}^{n \times n}$

5. 将非零项转成 `LayerGraph.inter_edges`，至少包含：

`src_unit, dst_unit, edge_type='cci', cci_score_raw, cci_score_norm, influence_score`

3.2 对 gene 层读取 GRN 图

如果 layer 是 gene 或后续约定的 gene-level layer：

1. 不读取 CCI。
2. 读取 gene x gene GRN。优先检查项目已有的 GRN 文件，例如：

`grn/<layer>/<sample>/grn_edges.csv`

如果实际 sample data 中 gene 层 GRN 路径不同，Codex 必须根据 sample data 适配，不要硬编码错误路径。

3. GRN edge table 至少需要：

`regulator, target, weight`

4. gene 层节点集合：

$G_{\{c,t\}} = \text{regulators} \cup \text{targets}$

5. gene 层邻接矩阵：

$A_{\text{gene}}(a,b) = \text{abs}(\text{weight}_{\{a,b\}})$

或者在配置项中允许选择 `abs / positive_only`。第一版默认 `abs`，避免 signed Laplacian 问题。

6. gene 层 `LayerGraph.intra_edges` 或 `inter_edges` 都可以，但 metadata 必须标清：

```
edge_source='grn'
uses_grn=True
uses_cci=False
layer_semantics='gene_grn'
```

3.3 LightCCI 的 NetworkContext 必须是 native units

LightCCI 不应该用 overlap/domain mapping 偷看上层结构来对齐 feature。它的 context 应设置：

```
feature_alignment_space = "native_units"
metadata["network_method"] = "light_cci"
metadata["feature_source"] = "light_cci_graph_only"
metadata["uses_expression_feature"] = False
```

但是 E compare feature 仍然可以在 `pij/compare/features.py` 中读取表达矩阵来构造表达特征。这不意味着 LightCCI network 本身使用 expression 作为边。

3.4 LightCCI 仍要提供节点、坐标、表达 fallback

为了让 compare 矩阵和主方法运行，NetworkContext 仍然需要：

- `lower_units_by_time`
- `upper_units_by_time`
- `lower_coords_by_time`
- `upper_coords_by_time`
- `lower_graphs`
- `upper_graphs`
- `shared_genes`

- `lower_mats / upper_mats` 可以为空特征矩阵, 例如 `np.zeros((n_units, 0))`

对于 `spot/domain` 层, 单位和坐标可来自对应 `.h5ad`。对于 `gene` 层, 如果没有空间坐标, 则坐标矩阵填零, 并在 `metadata` 写明 `has_spatial=False`。

4. 注册 LightCCI network

修改:

```
mignet_ce/networks/registry.py  
mignet_ce/config.py
```

4.1 registry.py

加入:

```
from mignet_ce.networks.light_cci import LightCCINetworkBuilder
```

并在 `NETWORK_BUILDERS` 中加入:

```
"light_cci": LightCCINetworkBuilder,
```

4.2 config.py

在 `NETWORK_METHODS` 中加入:

```
"light_cci",
```

如果用户需要在 `level pair` 中使用 `gene` 层, 还需要在 `LAYER_SPECS` 中加入 `gene` 层 spec。示例:

```
"gene": LayerSpec(name="gene", sample_prefix="gene", unit_kind="gene"),
```

但 Codex 不要盲目假设 `sample stem`, 一定要先看 `sample data` 或服务器实际数据。如果 `gene` 层没有 `h5ad`, `LightCCINetworkBuilder` 需要特殊处理 `gene layer`, 不要强制走普通 `LayerDataResolver` 的 `h5ad` 路径。

5. compare 特征必须使用真正的 LightCCI 图

当前 `mignet_ce/pij/compare/features.py` 中, `N` 和 `L` 的构建逻辑已经会读取 `COMMOT/CCI` 输出。但是这还不够, 因为:

- 如果 `network_method` 不是 `light_cci`, 上下文仍然是旧 `network`。
- 如果 `layer` 是 `gene`, `read_compare_adjacency()` 不能继续假设读取 `CCI`; 它必须使用 `LightCCI` 中的 `GRN adjacency`。

因此需要修改 `_adjacency_lists_for_side()`:

1. 如果 `context.network_method == "light_cci"`, 优先从 `context.lower_graphs / context.upper_graphs` 还原 `adjacency`。
2. 对 `CCI` 层, 从 `graph` 的 `CCI edges` 还原。
3. 对 `gene` 层, 从 `graph` 的 `GRN edges` 还原。
4. 只有非 `LightCCI context` 或 `graph adjacency` 缺失时, 才 fallback 到原来的 `read_compare_adjacency(paths, units, ...)`。

建议新增 helper:

```
def adjacency_from_lightcci_graph(graph: LayerGraph) -> sp.csr_matrix:  
    ...
```

它根据 `graph metadata` 判断:

```
edge_source == 'cci' -> 用 CCI edge
edge_source == 'grn' -> 用 GRN edge
```

这样 N 和 L 才真正吃的是 LightCCI network 产生的图，而不是在 PIJ 层绕开 network 自己重新找 CCI 文件。

6. 新增主方法文件：compare_main_lap_sr_spatial_sot.py

用户要求主方法放在 compare 下面的一个代码文件里。因此新增：

```
mignet_ce/pij/compare/compare_main_lap_sr_spatial_sot.py
```

注册方法名：

```
compare_main_lap_sr_spatial_sot
```

这个方法不属于 30 格消融矩阵，但放在 compare/ 下面，作为 LightCCI 主方法。

6.1 主方法数学定义

对 source 时间点 t_0 和 target 时间点 t_1 ，先从 LightCCI 图得到 Laplacian-HKS 特征：

$A_{\{t\}} \rightarrow L_{\{t\}} \rightarrow z_i^L$

其中：

- A_t 是 LightCCI adjacency。spot/domain 层来自 CCI；gene 层来自 GRN。
- L_t 是 normalized Laplacian。
- z_i^L 是节点 i 的 HKS 特征。

然后读取 SR 或发育状态量：

q_i, q'_j

再读取空间坐标：

r_i, r'_j

主方法预成本：

$$B_{ij} = (\lambda_L d_L(z_i^L, z_j'^L) + \lambda_{Sr} d_{Sr}(q_i, q'_j) + \lambda_R d_R(r_i, r'_j)) / (\lambda_L + \lambda_{Sr} + \lambda_R)$$

这里：

- B_{ij} 是 source 节点 i 到 target 节点 j 的预成本。
- d_L 默认使用 cosine distance。
- d_{Sr} 默认使用绝对差或标准化欧氏距离。
- d_R 默认使用空间欧氏距离，并做 robust normalization。
- λ_L 、 λ_{Sr} 、 λ_R 分别来自 config 中的权重。

候选集合：

$\Omega = \text{topK_source}(B) \cup \text{reverse_topK_target}(B)$

最终稀疏 OT 成本：

$C_{ij} = \text{normalize}(B_{ij}), (i, j) \in \Omega$

然后运行 sparse semi-relaxed OT：

$$\min_P \sum_{(i,j) \in \Omega} P_{ij} C_{ij} + \epsilon \sum_{(i,j) \in \Omega} P_{ij} (\log P_{ij} - 1) + \gamma \text{KL}(P || a)$$

s.t. $P^T \mathbf{1} = b$
 $P \geq 0$

```
supp(P) subset Omega
```

输出 raw transport 和 row-normalized PIJ。

6.2 主方法和 30 格矩阵的区别

必须在代码注释和 metadata 中写清楚：

- `compare_L_Sr_sot` 是矩阵格子，只使用当前 feature combination 的 distance。
 - `compare_main_lap_sr_spatial_sot` 是主方法，显式使用 Laplacian-HKS + SR + spatial 三项预成本。
 - 主方法不计入 30 格消融矩阵，但应与 30 格矩阵一起作为对照输出。
-

7. 注册主方法

修改：

```
mignet_ce/pij/registry.py  
mignet_ce/config.py
```

7.1 registry.py

加入 import：

```
from mignet_ce.pij.compare.compare_main_lap_sr_spatial_sot import  
↪ CompareMainLapSrSpatialSotPijMethod
```

在 `PIJ_METHOD_REGISTRY` 中加入：

```
"compare_main_lap_sr_spatial_sot": CompareMainLapSrSpatialSotPijMethod,
```

7.2 config.py

在 `PIJ_METHODS` 中加入：

```
"compare_main_lap_sr_spatial_sot"
```

建议新增 preset：

```
PIJ_METHOD_PRESETS["lightcci_main"] = ("compare_main_lap_sr_spatial_sot",)  
PIJ_METHOD_PRESETS["lightcci_all"] = (*COMPARE_PIJ_METHODS,  
↪ "compare_main_lap_sr_spatial_sot")
```

如果当前代码不方便动态改 dict，就直接在 `PIJ_METHOD_PRESETS` 字面量中补这两个 preset。

`DEVELOPMENT_PIJ_METHODS` 也要包含主方法，因为它需要 SR/development feature：

```
"compare_main_lap_sr_spatial_sot"
```

否则当用户没有传 `--development-feature-root` 时应该报错。

8. 修正 compare 公共文件结构漂移

当前 `code.zip` 里仍然有：

```
mignet_ce/pij/compare/distances.py
```

但前面已经要求 cosine 和 KL 分开。Codex 这次应顺手修正：

```
mignet_ce/pij/compare/cosine.py
mignet_ce/pij/compare/kl.py
mignet_ce/pij/compare/sparse_ot.py
mignet_ce/pij/compare/common.py
mignet_ce/pij/compare/features.py
```

要求：

- cosine distance 和 cosine kernel 放到 cosine.py。
 - KL divergence / JS 或 KL kernel 放到 kl.py。
 - sparse semi-relaxed OT 继续放到 sparse_ot.py。
 - common.py 只放调度、metadata、artifact、通用 shape check。
 - 不要在公共文件中新建 NMF 专属模块；N feature 的 temporal joint NMF 逻辑可以保留在 features.py 中，但不要调用 legacy joint_nmf.py。
-

9. 必须补 profile 和 sanity check

9.1 LightCCI network sanity check

实现后必须能通过：

```
python - <<'PY'
from mignet_ce.config import NETWORK_METHODS, PIJ_METHODS
from mignet_ce.networks.registry import NETWORK_BUILDERS
from mignet_ce.pij.registry import PIJ_METHOD_REGISTRY

assert 'light_cci' in NETWORK_METHODS
assert 'light_cci' in NETWORK_BUILDERS
assert 'compare_main_lap_sr_spatial_sot' in PIJ_METHODS
assert 'compare_main_lap_sr_spatial_sot' in PIJ_METHOD_REGISTRY
print('LightCCI network and main PIJ method are registered.')
PY
```

9.2 本机 sample data 快速检查

Codex 应写一个小检查或在 profile 中加入：

- 每个 layer/time 是否能找到 CCI total。
- CCI total shape 是否等于 index 行数。
- LightCCI graph 的 node 数、edge 数。
- gene layer 是否能找到 GRN，不能找到时给出明确报错。
- compare N/L 是否使用了 context.network_method == 'light_cci' 的 graph adjacency，而不是绕过 network 读取旧路径。

9.3 profile 脚本更新

修改或扩展：

scripts/profile_lightcci_compare_matrix.py

增加：

- --network-method light_cci 检查。
 - 输出每个 layer 的 edge_source: cci or grn。
 - 输出主方法候选边规模：|S|K_s + |T|K_t。
 - 输出主方法 dense 预成本内存估计、blockwise 计算建议、sparse OT 内存估计。
-

10. 建议运行命令

10.1 先跑主方法 profile

```
python scripts/profile_lightcci_compare_matrix.py \
--data-root /home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory \
--output-root /home/jovyan/work/2026\ Causality/output/profile_lightcci_main \
--organs heart \
--time-points 11.5 12.5 13.5 14.5 \
--level-pairs spot:seurat_k40 seurat_k40:seurat_k150 \
--network-method light_cci \
--pij-method compare_main_lap_sr_spatial_sot \
--development-feature-root /home/jovyan/public/datasets/Mouse-
  ↪ embryo/E1S1_domain_factory/development_features \
--profile-only \
--progress
```

10.2 再正式跑主方法

```
python scripts/run_mignet_vertical_ablation.py \
--data-root /home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory \
--output-root /home/jovyan/work/2026\
  ↪ Causality/output/lightcci_main_lap_sr_spatial_sot \
--organs heart \
--time-points 11.5 12.5 13.5 14.5 \
--level-pairs spot:seurat_k40 seurat_k40:seurat_k150 \
--network-method light_cci \
--pij-method compare_main_lap_sr_spatial_sot \
--development-feature-root /home/jovyan/public/datasets/Mouse-
  ↪ embryo/E1S1_domain_factory/development_features \
--export-pij \
--export-pij-topk 0 \
--progress \
--progress-log /home/jovyan/work/2026\
  ↪ Causality/output/lightcci_main_lap_sr_spatial_sot/progress.jsonl
```

10.3 再跑 30 格矩阵

30 格矩阵也必须使用：

```
--network-method light_cci
```

不能再用旧 network method 冒充 LightCCI。

11. 验收标准

Codex 完成后，必须满足：

1. --network-method light_cci 不再报 unsupported network method。
2. mignet_ce/networks/light_cci.py 存在，并且注册进 NETWORK_BUILDERS。
3. LightCCI 对 spot/domain 层读取 CCI，对 gene 层读取 GRN。
4. compare 的 N/L feature 在 network_method == 'light_cci' 时优先使用 LightCCI graph adjacency。
5. compare_main_lap_sr_spatial_sot 存在于 mignet_ce/pij/compare/，并注册进 PIJ_METHOD_REGISTRY。
6. 主方法 metadata 明确写出：Laplacian-HKS + SR + spatial pre-cost + sparse semi-relaxed OT。
7. 30 格消融矩阵和主方法可以分别运行，不互相混淆。
8. profile 输出能告诉用户是否真的在使用 LightCCI network，而不是旧网络。

12. 最重要的提醒

当前 code.zip 不能直接视为已经支持 LightCCI。它只是有一批 compare PIJ 文件雏形，但 network 层没有 light_cci，因此不能保证用户实际跑的是 LightCCI 架构。

本次修改的优先级最高的是：

先补 light_cci network method，再补 compare_main_lap_sr_spatial_sot 主方法。

如果这两件事没有完成，后续 30 格 compare 矩阵即使能跑，也只是跑在旧 network context 上，不能作为 LightCCI 实验结果。